What's new (2026-08-02)

- A link to your project's repo (`repo_url`) is now required for **every** submission, not just live-tier ones.
- Your submission zip must contain **exactly** the required files, nothing else — no source code, no `.git/` folder. The most common mistake: zipping your whole project folder by accident.
- Poster A3 size and demo video length/resolution are now checked **automatically** and will get your submission rejected, not just flagged for review.
- New self-check tool: `check_submission.sh` (Docker-based, nothing to install but Docker itself) — run it against your zip before you submit to catch problems early.
- Clarified: if your project also runs live elsewhere with real persistent data, keep `docker-compose.yml` demo-safe and put your real configuration under a different filename — see "The real code" below.

Show Off Your Project: Get Featured in the Gallery

Your project deserves an audience beyond the grader. A few minutes of packaging now gets you a shareable portfolio piece for LinkedIn, interviews, and your family group chat.

The Project Gallery is where we show projects from course **094290** ("Data collection and analysis") and **095219** ("Software for Machine Learning").

Currently the gallery is only for 094290. Once I have the data from 095219, it will have its own page.

We (The Course staff) want YOUR project to have a good-looking home. You'll be able to show it to your family and future employers.

What is needed from you (all in English)

- A short description of the project (~ 20 words)
- A long description (~ 100 words)
- An image (PNG/PDF) of size A3 (poster size) describing your system.

  > NOTE: the posters you see in the gallery now are TOO crowded. Just explain what the system does and give a few screenshots. (Tip: ask an AI tool to check that your font/color choices are accessible!)
  >
  > A3 size is checked automatically (as of 2026-08-02) — exactly for a PDF (page geometry is exact), by proportions only for a PNG/JPG (we can't reliably read a raster image's physical size). Run `check_submission.sh` (below) before you submit to catch this ahead of time.

- Your full names (the way you want the world to see it)
- A link to your project's repo (GitHub, GitLab, self-hosted, wherever — just needs to be a real URL). This is required for every submission now, not just projects with a live demo.

You do **not** need to make a separate thumbnail — we generate the small preview shown on the projects grid automatically from your poster.

Submission format

Submit **one zip file** in Moodle, containing a `metadata.json` file plus your poster (and video, if you have one). We build the gallery site from this data automatically, so please stick to the exact file names below.

```
<ID_number_of_one_of_the_group>.zip
├── metadata.json
├── poster.pdf        (or poster.png)
└── demo.mp4           (optional)
```

Your zip must contain exactly these files and nothing else — no source code, no `.git/` folder, no extra assets. The most common mistake here is zipping your whole project folder instead of just these files — double check before you upload. (Common zip-tool byproducts like macOS's `__MACOSX/` folder or `.DS_Store` are fine, we ignore those specifically — it's everything else that isn't.)

`metadata.json`:

```
{
  "title": "Your project title",
  "short_description": "~20 words describing the project",
  "long_description_md": "~100 words, markdown allowed",
  "team": [
    { "name": "Full Name 1" },
    { "name": "Full Name 2" }
  ],
  "tags": ["tag1", "tag2"],
  "tech_stack": ["React", "Python", "..."],
  "poster": { "type": "pdf" },
  "repo_url": "https://github.com/you/your-repo",
  "links": [
    { "label": "Demo Video", "url": "https://..." }
  ],
  "launch": {
    "type": "video | live | none",
    "notes": "e.g. seeded data / sample user, if the system runs live"
  }
}
```

Notes:

- Name the zip file after the ID number of one of the group members, e.g. `123456789.zip`.
- `poster.type` is `"pdf"` or `"image"` depending on which file you included.
- `repo_url` is required for every submission (any host — GitHub, GitLab, self-hosted, ...), separate from `links[]`, which is just for optional extra links shown on your page (demo site, video, etc).
- We (the staff) fill in `id`, `semester`, `year`, and the final file paths when we import your submission — you don't need to worry about those.
- If you're linking to a video hosted elsewhere (YouTube, Drive) instead of attaching `demo.mp4`: for now this is stored as a plain link, not wired into the video player — the gallery's video button currently only plays an attached

`demo.mp4`. Attach the file if you want the in-page "Play Demo Video" button to work.

Before you upload, check your submission. You need Docker installed — nothing else, no Python setup:

1. Download `check_submission.sh` (in `check-submission-docker/` in the course repo).
2. Run it against the zip you're about to submit:

```
bash check_submission.sh path/to/your-submission.zip
```

The first run downloads the checker automatically (a few seconds); after that it's instant. It runs the exact same checks the course staff's import tool does, so you find out about problems now instead of after the deadline.

Advanced stuff: let's see your project run for real

Now this is more complicated. Imagine a user clicks on your project's thumbnail, and choose 'Launch Project'.

What do you think should happen? a demo video? run the actual system?

If the system runs, what data does it use (i.e. seeded data, sample user)

There are other considerations for me to handle: where to run, how to deploy, how to start, for how long etc.

Demo video

If you can provide a screen recording or similar:

Show how the system is used.

- Video length between 20 and 90 seconds
- Resolution: 1920*1080
- Format: MP4
- Silent audio (or no audio)

Length and resolution are checked automatically (as of 2026-08-02) — a `demo.mp4` outside 20–90s or not exactly 1920×1080 will be rejected, not just flagged for review. `check_submission.sh` (above) catches this before you submit. The silent-audio guideline isn't automatically checked.

The real code

If you can supply us with a well-packaged container setup, this is the real deal — visitors get to run your actual system, not just watch a video of it.

- Your `repo_url` (see above — required for every submission) needs to actually be reachable for this part specifically, so we can clone it.
- The repo **must contain a `docker-compose.yml`** that follows our Compose rules (`docs/reference/compose-contract.md` in the course repo) — no volumes, no privileged mode, no host networking, every service must declare `mem_limit` and `cpus`, and **exactly one service must publish a `ports:` mapping** (that's the one

visitors will reach). See `example-projects/` in the course repo for two working examples (a Flask+Mongo and a Flask+Postgres app).

- Multi-service setups (e.g. app + database) are supported — each service becomes its own container. **Do not rely on `depends_on`** to sequence startup: your compose file is only used to build and validate images, not to actually run `docker compose up` — the runtime is Azure Container Instances, which has no `depends_on`-equivalent ordering. If your app needs its database to be up first, you must implement that wait/retry yourself (e.g. retry the connection on startup until it succeeds) — don't assume the database container is already up.
- Every service must seed its own data from code on startup — no persistent volumes are allowed, so a restarted demo always starts from the same clean state.
- We no longer accept a plain `installation.md` / `install.sh` as an alternative — the live-tier pipeline only builds from `docker-compose.yml`.
- **If your project is genuinely live elsewhere with real persistent data** (real accounts, real user content — not just a local dev setup): the `docker-compose.yml` at your repo root still has to be the demo-safe one (no volumes, seeds its own data, kiosk mode — same rules as above) — that's the only one the gallery ever builds or runs. Your real deployment's actual configuration, with real persistence, can live in the same repo under a different name (e.g. `docker-compose.production.yml`) — we simply never touch it. **Note this isn't checked automatically**: nothing validates that a second file exists or that it's the "real" one — it's on you to keep `docker-compose.yml` itself demo-safe regardless of what else is in the repo.
- If your repo genuinely can't be public (e.g. a real live service with a private codebase), that's fine for the top-level `repo_url` requirement — it just needs to be a real URL, not necessarily one we can clone. It only becomes a live-tier problem specifically (see above): if we can't clone it, you won't get a live "run it yourself" button, but the rest of your submission (poster, description, video) still goes through normally.